

PracticoEne22.pdf



andreagb03



Inteligencia Artificial



3º Grado en Ingeniería Informática - Tecnologías Informáticas



Escuela Técnica Superior de Ingeniería Informática
Universidad de Sevilla



[Accede al documento original](#)

INTERCAMBIA y AHORRA en LA VUELTA A LA UNI

5 AÑOS DE GARANTÍA en todo

CeX
webuy.com

$$6+7=$$



**CURSO
NUEVO
EQUIPO
NUEVO**

27/12/24, 12:19

Practico_Ene22

Inteligencia Artificial

Examen Final - Parte Práctica

23 de Enero, 2021

```
In [2]: # Nombre y apellidos:  
# Aula:  
# Puesto:
```

```
In [3]: import math, random
```

En los siguientes ejercicios implementaremos algunas funciones relacionadas con el algoritmo de clasificación kNN (K-nearest neighbors) y su evaluación sobre conjuntos de ejemplos (datasets). Representaremos un ejemplo como un vector de n valores numéricos donde las primeras $n - 1$ posiciones representan los valores de sus atributos y la última posición representa el valor de clasificación asociado a dicho ejemplo. Para simplificar, consideraremos que todos los atributos y los valores de clasificación asociados a los ejemplos toman valores numéricos. Para las pruebas utilizaremos los siguientes conjuntos de ejemplos:

```
In [7]: dataset1 = [[2.7810836, 2.550537003, 0],  
[1.465489372, 2.362125076, 0],  
[3.396561688, 4.400293529, 0],  
[1.38807019, 1.850220317, 0],  
[3.06407232, 3.005305973, 0],  
[7.627531214, 2.759262235, 1],  
[5.332441248, 2.088626775, 1],  
[6.922596716, 1.77106367, 1],  
[8.675418651, -0.242068655, 1],  
[7.673756466, 3.508563011, 1]]  
  
dataset2 = [  
[-1, -1, 1, -1, -1, 1, 1, 1, 1, 1, 1, -1, 1, 1, -1, 1],  
[1, -1, 1, -1, -1, -1, 1, 1, 1, -1, -1, -1, -1, -1, 1, 0],  
[1, 1, 1, -1, -1, 1, 0, 1, -1, -1, 1, 1, -1, 1, -1, 0],  
[1, -1, 1, 1, -1, -1, -1, 1, 1, 1, -1, -1, -1, 1, 1, 1],  
[0, 0, 0, -1, -1, -1, 1, 1, 1, 1, -1, -1, 1, -1, 1, 1],  
[-1, 1, 1, -1, -1, -1, 1, 1, 1, -1, -1, -1, 1, -1, 0, 0],  
[1, -1, 1, -1, -1, 0, 1, 1, 1, -1, 0, 0, -1, 0, 0, 0],  
[-1, -1, -1, 1, 1, 1, -1, -1, -1, -1, -1, 1, 1, 1, -1, -1],  
[-1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0],  
[1, -1, -1, 1, 1, 1, -1, -1, -1, -1, 1, 1, 1, 1, -1, -1],  
[-1, 1, -1, 1, 1, -1, -1, -1, -1, -1, 0, 0, 1, 1, -1, -1],  
[-1, -1, 1, -1, -1, -1, 1, 1, 1, -1, -1, -1, -1, 1, 1, 1],  
[-1, 1, 1, -1, -1, 1, 1, 1, -1, 1, -1, -1, -1, -1, 1, 1],  
[1, 1, -1, -1, 1, 1, -1, -1, -1, 1, 1, 1, 1, 1, -1, 0],  
[-1, 1, -1, 1, 1, 1, -1, 0, -1, 1, -1, 1, 1, 1, -1, 0],  
[-1, 1, -1, 1, 1, 1, -1, -1, -1, 1, -1, 1, 1, 1, -1, -1]]  
  
dataset3 = [[5.1, 3.5, 1.4, 0],  
[4.9, 3.0, 1.4, 0],
```



WUOLAH

```
[4.7, 3.2, 1.3, 0],
[4.6, 3.1, 1.5, 0],
[5.0, 3.6, 1.4, 0],
[5.4, 3.9, 1.7, 0],
[4.6, 3.4, 1.4, 0],
[5.0, 3.4, 1.5, 0],
[4.4, 2.9, 1.4, 0],
[4.9, 3.1, 1.5, 0],
[7.0, 3.2, 4.7, 1],
[6.4, 3.2, 4.5, 1],
[6.9, 3.1, 4.9, 1],
[5.5, 2.3, 4.0, 1],
[6.5, 2.8, 4.6, 1],
[5.7, 2.8, 4.5, 1],
[6.3, 3.3, 4.7, 1],
[4.9, 2.4, 3.3, 1],
[6.6, 2.9, 4.6, 1],
[5.2, 2.7, 3.9, 1],
[5.0, 2.0, 3.5, 1],
[6.3, 3.3, 6.0, 2],
[5.8, 2.7, 5.1, 2],
[7.1, 3.0, 5.9, 2],
[6.3, 2.9, 5.6, 2],
[6.5, 3.0, 5.8, 2],
[7.6, 3.0, 6.6, 2],
[4.9, 2.5, 4.5, 2],
[7.3, 2.9, 6.3, 2],
[6.7, 2.5, 5.8, 2],
[7.2, 3.6, 6.1, 2]]
```

Ejercicio 1 [0,75 ptos]:

Implementa las siguientes tres funciones de distancia que podrán ser utilizadas por el método kNN:

- **distancia_euclidea(v1,v2)**, cuya definición se presenta a continuación.

$$d_E(P, Q) = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}$$

- **distancia_manhattan(v1,v2)**, cuya definición se presenta a continuación.

$$d_M(P, Q) = \sum_{i=1}^n |p_i - q_i|$$

- **distancia_hamming(v1,v2)**, que se define como el número de atributos en los que difieren dos ejemplos dados.

Nota: Estas funciones recibirán dos vectores **v1** y **v2** de igual longitud que representan los valores de los atributos de dos ejemplos **sin sus valores de clasificación**.

```
In [43]: ## Solución:
def distancia_euclidea(v1,v2):
    return math.sqrt(sum((pi-qi)**2 for pi, qi in zip(v1,v2)))
```

```
def distancia_manhattan(v1,v2):
    return sum(abs(pi-qi) for pi, qi in zip(v1,v2))

def distancia_hamming(v1,v2):
    return sum(1 for pi, qi in zip(v1,v2) if pi!=qi)
```

Puedes probar tu solución con los siguientes ejemplos:

```
In [46]: distancia_euclidea([1.465489372,2.362125076], [3.396561688,4.400293529])
#Salida esperada: 2.80769851166859
```

```
Out[46]: 2.80769851166859
```

```
In [48]: distancia_euclidea([-1, -1, 1, -1], [1, -1, 1, -1])
#Salida esperada: 2.0
```

```
Out[48]: 2.0
```

```
In [50]: distancia_manhattan([1.465489372,2.362125076], [3.396561688,4.400293529])
#Salida esperada: 3.969240769
```

```
Out[50]: 3.969240769
```

```
In [52]: distancia_manhattan([-1, -1, 1, -1], [1, -1, 1, -1])
#Salida esperada: 2
```

```
Out[52]: 2
```

```
In [54]: distancia_hamming([1.465489372,2.362125076], [3.396561688,4.400293529])
#Salida esperada: 2
```

```
Out[54]: 2
```

```
In [56]: distancia_hamming([-1, -1, 1, -1], [1, -1, 1, -1])
#Salida esperada: 1
```

```
Out[56]: 1
```

Ejercicio 2 [0,75 pts]

Implementa una función **vecinos(distancia,dataset,ejemplo,k)** que dada una función de **distancia**, un **dataset**, un **ejemplo** y un valor entero **k** devuelva los **k** ejemplos en **dataset** más cercanos a **ejemplo** según la función **distancia**.

```
In [66]: ## Solución:
def vecinos(distancia, dataset, ejemplo, k):
    distancias=[(fila, distancia(ejemplo, fila[:-1])) for fila in dataset]
    distancias_ordenadas=sorted(distancias, key=lambda x: x[1])
    return [vecino[0] for vecino in distancias_ordenadas[:k]]
```

Ejemplos de uso:

```
In [69]: vecinos(distancia_euclidea,dataset1,[1,2],3)
#Salida esperada:
[[1.38807019, 1.850220317, 0],
```

6+7=



CURSO
NUEVO
EQUIPO
NUEVO

27/12/24, 12:19

Practico_Ene22

```
[1.465489372, 2.362125076, 0],  
[2.7810836, 2.550537003, 0]]
```

```
Out[69]: [[1.38807019, 1.850220317, 0],  
[1.465489372, 2.362125076, 0],  
[2.7810836, 2.550537003, 0]]
```

```
In [71]: vecinos(distancia_manhattan,dataset1,[7,-1],3)  
#Salida esperada:  
#[[8.675418651, -0.242068655, 1],  
# [6.922596716, 1.77106367, 1],  
# [7.627531214, 2.759262235, 1]]
```

```
Out[71]: [[8.675418651, -0.242068655, 1],  
[6.922596716, 1.77106367, 1],  
[7.627531214, 2.759262235, 1]]
```

```
In [73]: vecinos(distancia_hamming,dataset2,[1, -1, 1, -1, -1, 1, 1, 1, 1, 1, 1, 1, -1, 1, 1]  
#Salida esperada:  
#[[-1, -1, 1, -1, -1, 1, 1, 1, 1, 1, 1, 1, -1, 1, 1],  
# [1, -1, 1, -1, -1, 1, 1, 1, 1, -1, -1, -1, -1, 1, 0],  
# [1, 1, 1, -1, -1, 1, 0, 1, -1, -1, 1, 1, -1, 1, -1, 0]]
```

```
Out[73]: [[-1, -1, 1, -1, -1, 1, 1, 1, 1, 1, 1, 1, -1, 1, 1],  
[1, -1, 1, -1, -1, 1, 1, 1, 1, -1, -1, -1, -1, 1, 0],  
[1, 1, 1, -1, -1, 1, 0, 1, -1, -1, 1, 1, -1, 1, -1, 0]]
```

```
In [75]: vecinos(distancia_hamming,dataset3,[6.5, 2.8, 4.4],3)  
#Salida esperada:  
#[[6.5, 2.8, 4.6, 1], [5.7, 2.8, 4.5, 1], [6.5, 3.0, 5.8, 2]]
```

```
Out[75]: [[6.5, 2.8, 4.6, 1], [5.7, 2.8, 4.5, 1], [6.5, 3.0, 5.8, 2]]
```

Ejercicio 3 [0.5 ptos]:

A continuación, define una función **clasifica_kNN(distancia,dataset,ejemplo,k)** que recibiendo una función **distancia**, un **dataset**, un **ejemplo** y un valor entero **k** devuelva el valor de clasificación mayoritario en los **k** vecinos en **dataset** más cercanos a **ejemplo** según la función **distancia** recibida. Se espera que el valor de **k** a recibir sea impar para evitar empates.

```
In [88]: ## Solución:  
def clasifica_kNN(distancia,dataset,ejemplo,k):  
    k_vecinos = vecinos(distancia, dataset, ejemplo, k)  
    clasificaciones = [vecino[-1] for vecino in k_vecinos]  
    return max(set(clasificaciones), key=clasificaciones.count)
```

Ejemplos de uso:

```
In [91]: clasifica_kNN(distancia_euclidea,dataset1,[1,-1],3)  
#Salida esperada: 0
```

```
Out[91]: 0
```

```
In [93]: clasifica_kNN(distancia_manhattan,dataset1,[7,2],3)  
#Salida esperada: 1
```

WUOLAH

Out[93]: 1

In [95]: `clasifica_kNN(distancia_hamming,dataset2,[1, -1, 1, -1, -1, 1, 1, 1, 1, 1, 1, -1])`
#Salida esperada: 0

Out[95]: 0

In [97]: `clasifica_kNN(distancia_hamming,dataset2,[-1, -1, 1, -1, -1, -1, 1, 1, 1, -1, -1])`
#Salida esperada: 1

Out[97]: 1

In [99]: `clasifica_kNN(distancia_hamming,dataset2,[-1, -1, -1, 1, 1, 1, -1, -1, -1, -1, -1])`
#Salida esperada: -1

Out[99]: -1

In [101... `clasifica_kNN(distancia_euclidea,dataset3,[6.5, 2.8, 4.4],3)`
#Salida esperada: 1

Out[101... 1

In [103... `clasifica_kNN(distancia_manhattan,dataset3,[6.5, 2.8, 4.4],1)`
#Salida esperada: 1

Out[103... 1

Ejercicio 4 [0,75 pts]:

Ya que los conjuntos de datos con los que estamos trabajando son pequeños, realizaremos una evaluación del método utilizando validación cruzada. Implementa una función **kfolds(dataset,k)** que recibe un **dataset** y un valor entero **k** y divide el **dataset** en **k** subconjuntos, de tal manera que cada ejemplo del **dataset** debe pertenecer a sólo uno de los subconjuntos. La distribución de los ejemplos en los diferentes subconjuntos se realizará de manera aleatoria. Siempre que sea posible, los subconjuntos obtenidos deben tener la misma cardinalidad (esto no será posible cuando el número de ejemplos en el dataset no sea múltiplo de **k**).

In [106... *## Solución:*

```
def kfolds(dataset,k):
    #mezclamos aleatoriamente el dataset
    datos=dataset[:]
    random.shuffle(datos)
    #dividimos los datos en k subconjuntos
    fold_size=len(datos)//k
    folds=[]
    for i in range(k):
        start_idx=i*fold_size
        #último fold toma los elementos restantes
        if i==k-1:
            folds.append(datos[start_idx:])
        else:
            folds.append(datos[start_idx:start_idx+fold_size])
    return folds
```

6 + 7 =

**CURSO
NUEVO
EQUIPO
NUEVO**



**5
AÑOS
DE GARANTÍA
en todo**



webuy.com



```
In [108... kfold(dataset1,3)
#Salida esperada (puede variar):
#[[3.06407232, 3.005305973, 0],
# [8.675418651, -0.242068655, 1],
# [2.7810836, 2.550537003, 0],
# [6.922596716, 1.77106367, 1]],
# [[1.38807019, 1.850220317, 0],
# [5.332441248, 2.088626775, 1],
# [7.627531214, 2.759262235, 1]],
# [[1.465489372, 2.362125076, 0],
# [7.673756466, 3.508563011, 1],
# [3.396561688, 4.400293529, 0]]]
```

```
Out[108... [[1.38807019, 1.850220317, 0],
[3.06407232, 3.005305973, 0],
[1.465489372, 2.362125076, 0]],
[[3.396561688, 4.400293529, 0],
[7.673756466, 3.508563011, 1],
[5.332441248, 2.088626775, 1]],
[[8.675418651, -0.242068655, 1],
[7.627531214, 2.759262235, 1],
[6.922596716, 1.77106367, 1],
[2.7810836, 2.550537003, 0]]]
```

```
In [110... kfold(dataset1,2)
#Salida esperada (puede variar):
#[[7.627531214, 2.759262235, 1],
# [1.38807019, 1.850220317, 0],
# [8.675418651, -0.242068655, 1],
# [7.673756466, 3.508563011, 1],
# [3.06407232, 3.005305973, 0]],
# [[5.332441248, 2.088626775, 1],
# [6.922596716, 1.77106367, 1],
# [3.396561688, 4.400293529, 0],
# [2.7810836, 2.550537003, 0],
# [1.465489372, 2.362125076, 0]]]
```

```
Out[110... [[1.38807019, 1.850220317, 0],
[1.465489372, 2.362125076, 0],
[8.675418651, -0.242068655, 1],
[3.396561688, 4.400293529, 0],
[7.673756466, 3.508563011, 1]],
[[6.922596716, 1.77106367, 1],
[2.7810836, 2.550537003, 0],
[3.06407232, 3.005305973, 0],
[5.332441248, 2.088626775, 1],
[7.627531214, 2.759262235, 1]]]
```

```
In [112... kfold(dataset1,10)
#Salida esperada (puede variar):
#[[6.922596716, 1.77106367, 1]],
# [[7.627531214, 2.759262235, 1]],
# [[7.673756466, 3.508563011, 1]],
# [[3.396561688, 4.400293529, 0]],
# [[2.7810836, 2.550537003, 0]],
# [[8.675418651, -0.242068655, 1]],
# [[1.465489372, 2.362125076, 0]],
# [[5.332441248, 2.088626775, 1]],
```

27/12/24, 12:19

Practico_Ene22

```
# [[1.38807019, 1.850220317, 0]],
# [[3.06407232, 3.005305973, 0]]]
```

```
Out[112... [[1.38807019, 1.850220317, 0]],
[[8.675418651, -0.242068655, 1]],
[[5.332441248, 2.088626775, 1]],
[[3.06407232, 3.005305973, 0]],
[[3.396561688, 4.400293529, 0]],
[[7.673756466, 3.508563011, 1]],
[[7.627531214, 2.759262235, 1]],
[[1.465489372, 2.362125076, 0]],
[[2.7810836, 2.550537003, 0]],
[[6.922596716, 1.77106367, 1]]]
```

```
In [114... kfold(dataset2,3)
#Salida esperada (puede variar):
#[[-1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0],
# [-1, 1, -1, 1, 1, -1, -1, -1, -1, -1, 0, 0, 1, 1, -1, -1],
# [-1, 1, 1, -1, -1, 1, 1, 1, -1, 1, -1, -1, -1, 1, 1],
# [1, 1, -1, -1, 1, 1, -1, -1, -1, 1, 1, 1, 1, 1, -1, 0],
# [-1, 1, -1, 1, 1, 1, -1, 0, -1, 1, -1, 1, 1, 1, -1, 0],
# [1, -1, 1, -1, -1, 0, 1, 1, 1, -1, 0, 0, -1, 0, 0, 0]],
# [-1, -1, -1, 1, 1, 1, -1, -1, -1, -1, -1, 1, 1, 1, -1, -1],
# [-1, 1, 1, -1, -1, -1, 1, 1, 1, -1, -1, -1, 1, -1, 0, 0],
# [-1, 1, -1, 1, 1, 1, -1, -1, -1, 1, -1, 1, 1, 1, -1, -1],
# [0, 0, 0, -1, -1, -1, 1, 1, 1, 1, -1, -1, 1, -1, 1, 1],
# [-1, -1, 1, -1, -1, -1, 1, 1, 1, -1, -1, -1, -1, 1, 1, 1]],
# [-1, -1, 1, -1, -1, 1, 1, 1, 1, 1, -1, 1, 1, -1, 1],
# [1, 1, 1, -1, -1, 1, 0, 1, -1, -1, 1, 1, -1, 1, -1, 0],
# [1, -1, 1, 1, -1, -1, -1, 1, 1, 1, -1, -1, -1, 1, 1, 1],
# [1, -1, 1, -1, -1, -1, 1, 1, 1, -1, -1, -1, -1, 1, 0],
# [1, -1, -1, 1, 1, 1, -1, -1, -1, -1, 1, 1, 1, 1, -1, -1]]]
```

```
Out[114... [[0, 0, 0, -1, -1, -1, 1, 1, 1, 1, -1, -1, 1, -1, 1, 1],
[1, -1, -1, 1, 1, 1, -1, -1, -1, -1, 1, 1, 1, 1, -1, -1],
[1, 1, 1, -1, -1, 1, 0, 1, -1, -1, 1, 1, -1, 1, -1, 0],
[-1, 1, 1, -1, -1, -1, 1, 1, 1, -1, -1, -1, 1, -1, 0, 0],
[-1, -1, -1, 1, 1, 1, -1, -1, -1, -1, -1, 1, 1, 1, -1, -1]],
[[-1, 1, -1, 1, 1, 1, -1, -1, -1, 1, -1, 1, 1, 1, -1, -1],
[1, -1, 1, -1, -1, -1, 1, 1, 1, -1, -1, -1, -1, -1, 1, 0],
[-1, 1, 1, -1, -1, 1, 1, 1, 1, -1, 1, -1, -1, -1, 1, 1],
[-1, -1, 1, -1, -1, 1, 1, 1, 1, 1, 1, 1, -1, 1, 1, 1],
[1, -1, 1, 1, -1, -1, -1, 1, 1, 1, -1, -1, -1, 1, 1, 1]],
[[-1, -1, 1, -1, -1, -1, 1, 1, 1, -1, -1, -1, -1, 1, 1, 1],
[1, 1, -1, -1, 1, 1, -1, -1, -1, 1, 1, 1, 1, 1, -1, 0],
[-1, 1, -1, 1, 1, -1, -1, -1, -1, 0, 0, 1, 1, -1, -1],
[-1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0],
[-1, 1, -1, 1, 1, 1, -1, 0, -1, 1, -1, 1, 1, 1, -1, 0],
[1, -1, 1, -1, -1, 0, 1, 1, 1, -1, 0, 0, -1, 0, 0, 0]]]
```

```
In [116... kfold(dataset3,4)
#Salida esperada (puede variar):
#[[7.0, 3.2, 4.7, 1],
# [5.8, 2.7, 5.1, 2],
# [6.5, 3.0, 5.8, 2],
# [4.9, 3.0, 1.4, 0],
# [6.4, 3.2, 4.5, 1],
# [7.1, 3.0, 5.9, 2],
# [5.0, 3.4, 1.5, 0],
# [4.6, 3.1, 1.5, 0]],
```

mola que
estés así
el sábado

pero mola
más que
estés así
el lunes.

```
# [[7.3, 2.9, 6.3, 2],
#  [6.3, 2.9, 5.6, 2],
#  [6.5, 2.8, 4.6, 1],
#  [5.0, 2.0, 3.5, 1],
#  [7.2, 3.6, 6.1, 2],
#  [6.7, 2.5, 5.8, 2],
#  [6.3, 3.3, 4.7, 1],
#  [6.9, 3.1, 4.9, 1]],
# [[6.3, 3.3, 6.0, 2],
#  [5.0, 3.6, 1.4, 0],
#  [5.2, 2.7, 3.9, 1],
#  [5.5, 2.3, 4.0, 1],
#  [4.6, 3.4, 1.4, 0],
#  [4.4, 2.9, 1.4, 0],
#  [4.7, 3.2, 1.3, 0],
#  [5.7, 2.8, 4.5, 1]],
# [[5.4, 3.9, 1.7, 0],
#  [6.6, 2.9, 4.6, 1],
#  [4.9, 2.5, 4.5, 2],
#  [4.9, 2.4, 3.3, 1],
#  [7.6, 3.0, 6.6, 2],
#  [5.1, 3.5, 1.4, 0],
#  [4.9, 3.1, 1.5, 0]]]
```

```
Out[116... [[6.9, 3.1, 4.9, 1],
            [7.6, 3.0, 6.6, 2],
            [6.7, 2.5, 5.8, 2],
            [5.1, 3.5, 1.4, 0],
            [4.6, 3.4, 1.4, 0],
            [4.4, 2.9, 1.4, 0],
            [7.0, 3.2, 4.7, 1]],
            [[6.4, 3.2, 4.5, 1],
            [5.2, 2.7, 3.9, 1],
            [7.1, 3.0, 5.9, 2],
            [5.4, 3.9, 1.7, 0],
            [5.5, 2.3, 4.0, 1],
            [4.9, 2.5, 4.5, 2],
            [5.7, 2.8, 4.5, 1]],
            [[6.3, 3.3, 6.0, 2],
            [4.9, 3.0, 1.4, 0],
            [6.6, 2.9, 4.6, 1],
            [6.5, 2.8, 4.6, 1],
            [4.9, 2.4, 3.3, 1],
            [7.2, 3.6, 6.1, 2],
            [6.3, 2.9, 5.6, 2]],
            [[4.9, 3.1, 1.5, 0],
            [4.6, 3.1, 1.5, 0],
            [5.0, 2.0, 3.5, 1],
            [7.3, 2.9, 6.3, 2],
            [5.8, 2.7, 5.1, 2],
            [5.0, 3.4, 1.5, 0],
            [5.0, 3.6, 1.4, 0],
            [6.5, 3.0, 5.8, 2],
            [4.7, 3.2, 1.3, 0],
            [6.3, 3.3, 4.7, 1]]]
```

Ejercicio 5 [0,75 pts]:

Por último, define una función **cross_val_kNN(distancia,dataset,k1,k2)** que recibiendo una función **distancia**, un **dataset**, un valor entero **k1** que define el parámetro k (número de vecinos a tener en cuenta) del algoritmo kNN y otro valor entero **k2** que define el número de folds en los que hay que dividir el conjunto de entrenamiento, realice lo siguiente:

1. Dividir el dataset en **k2** folds utilizando el método creado en el apartado anterior.
2. Para cada uno de los folds evaluar el rendimiento (la proporción de ejemplos que el método kNN clasifica correctamente) utilizando como datos de entrenamiento el resto de folds, el parámetro **k1** como número de vecinos a tener en cuenta y la función **distancia** como función de distancia para el método kNN.
3. Devolver la media de los rendimientos para los diferentes folds.

```
In [119... ## Solución:
def cross_val_kNN(distancia,dataset,k1,k2):
    # Dividimos el dataset en k2 folds
    folds = kfold(dataset, k2)
    rendimientos = []
    # Para cada fold, evaluamos el rendimiento
    for i in range(k2):
        # Fold actual se utiliza como conjunto de prueba
        conjunto_prueba = folds[i]
        # El resto de los folds forman el conjunto de entrenamiento
        conjunto_entrenamiento = [
            ejemplo for j, fold in enumerate(folds) if j != i for ejemplo in fol
        ]
        # Evaluamos el rendimiento en el conjunto de prueba
        aciertos = 0
        for ejemplo in conjunto_prueba:
            # Predicción del método kNN
            prediccion = clasifica_kNN(distancia, conjunto_entrenamiento, ejempl
            # Comprobamos si la predicción coincide con el valor real de clasifi
            if prediccion == ejemplo[-1]:
                aciertos += 1
        # Calculamos el rendimiento para este fold
        rendimiento = aciertos / len(conjunto_prueba)
        rendimientos.append(rendimiento)
    # Devolvemos la media de los rendimientos
    return sum(rendimientos) / len(rendimientos)
```

```
In [121... cross_val_kNN(distancia_euclidea,dataset1,3,10)
#Salida esperada (Los resultados pueden variar):
#1.0
```

```
Out[121... 1.0
```

```
In [123... cross_val_kNN(distancia_manhattan,dataset1,3,10)
#Salida esperada (Los resultados pueden variar):
#1.0
```

```
Out[123... 1.0
```

```
In [125... cross_val_kNN(distancia_hamming,dataset1,3,10)
#Salida esperada (Los resultados pueden variar):
#0.5
```

$$6+7=$$



**CURSO
NUEVO
EQUIPO
NUEVO**

27/12/24, 12:19

Practico_Ene22

Out[125...] 0.3

```
In [127...] cross_val_knn(distancia_euclidea,dataset2,3,10)
#Salida esperada (Los resultados pueden variar):
#0.5625
```

Out[127...] 0.37142857142857144

```
In [129...] cross_val_knn(distancia_manhattan,dataset3,3,2)
#Salida esperada (Los resultados pueden variar):
#0.8709677419354839
```

Out[129...] 0.9354166666666667

```
In [131...] cross_val_knn(distancia_manhattan,dataset3,3,10)
#Salida esperada (Los resultados pueden variar):
#0.9354838709677419
```

Out[131...] 0.9333333333333333



webuy.com



WUOLAH